Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Services

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

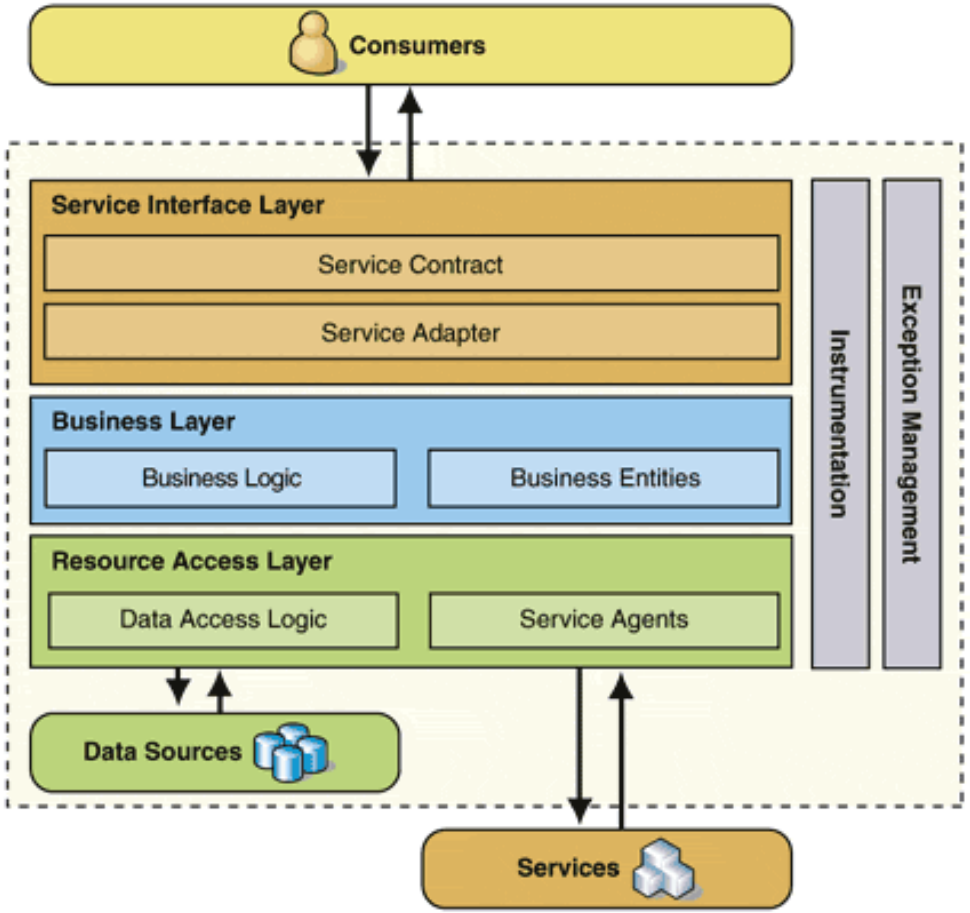
- Services & how services are used to make life easier
- Service Oriented Architectures (SOAs)
- Services in Business Applications and conventional layered applications
- Web Services (especially in .NET)
- The Web Service architecture

UNIVERSITY of HULL

Three layered service architecture

- Typical business **application** offering a service
- Layers separate logical groupings of functionality
  - Make it easier to change without side-effects
  - Assist with security
  - Each layer may exist on different/multiple computers

- A Service consumer interacts with a service through the Service Interface Layer
- Each service may have:
  - business logic/entities
  - data/data sources
  - links to other services that it uses



UNIVERSITY of HULL

Service Oriented Architecture (SOA)

“A loosely-coupled architecture designed to meet the business needs of the organisation”

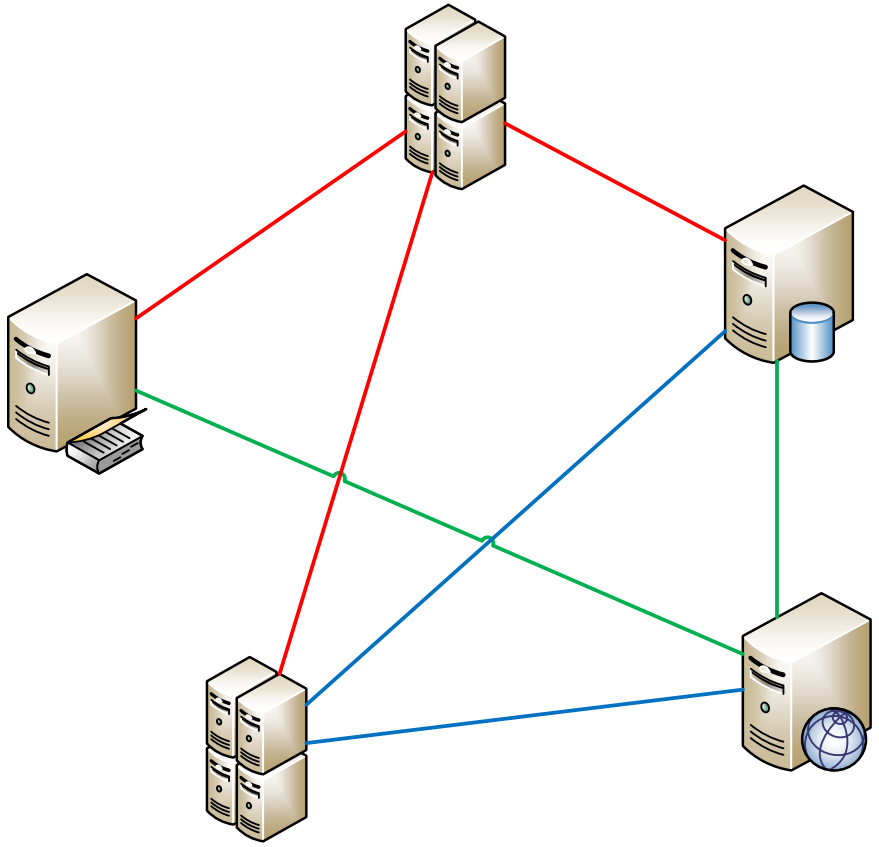
- Consider a business, with a number of different computer systems working together:

- *Each coloured line represents a different communication method between the components.*

- *In order to make a request, a component must know how to communicate with another component.*

- *Each component must be able to speak the language of all other components.*

- ***This means all components end up strongly coupled to each other. Replacing a component is difficult!***



UNIVERSITY of HULL

Service Oriented Architecture (SOA)

“A loosely-coupled architecture designed to meet the business needs of the organisation”

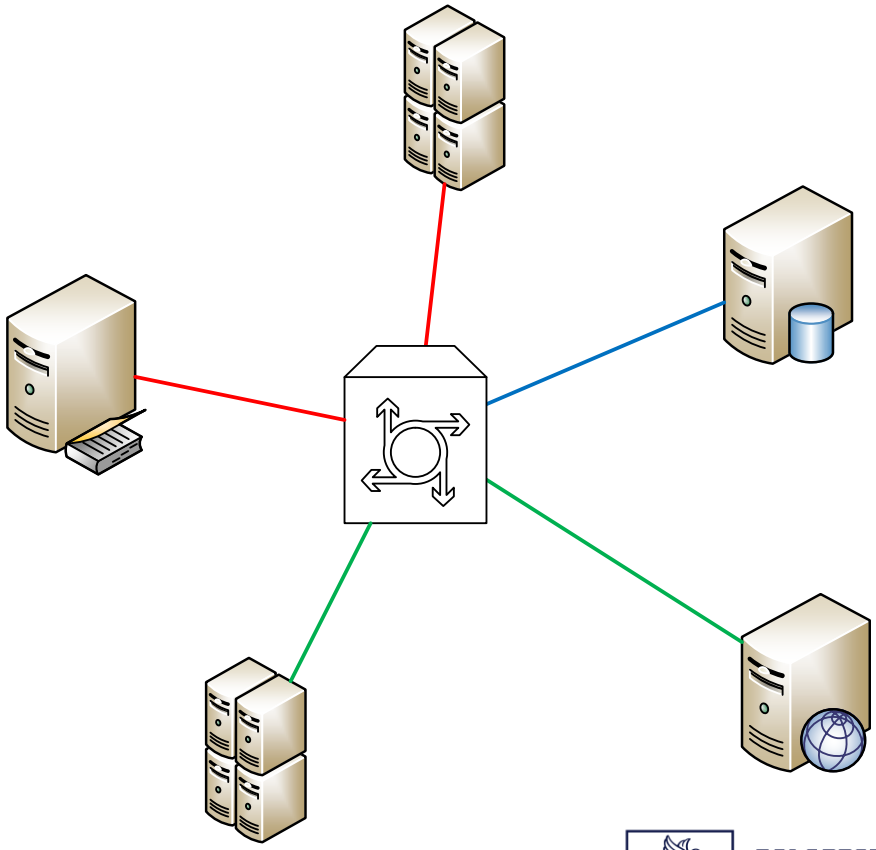
- Consider a business, with a number of different computer systems working together:

- *Now, the components communicate with a service bus which sits in between.*

- *Each component may talk to the service bus in a slightly different way – but only the service bus needs to be able to do the translation.*

- *Now, the components are loosely coupled. They can make requests through the bus whilst speaking only one language.*

- ***Now it’s really easy to replace a component!***



UNIVERSITY of HULL

Services

- Services offer functionality to be called from a single point.

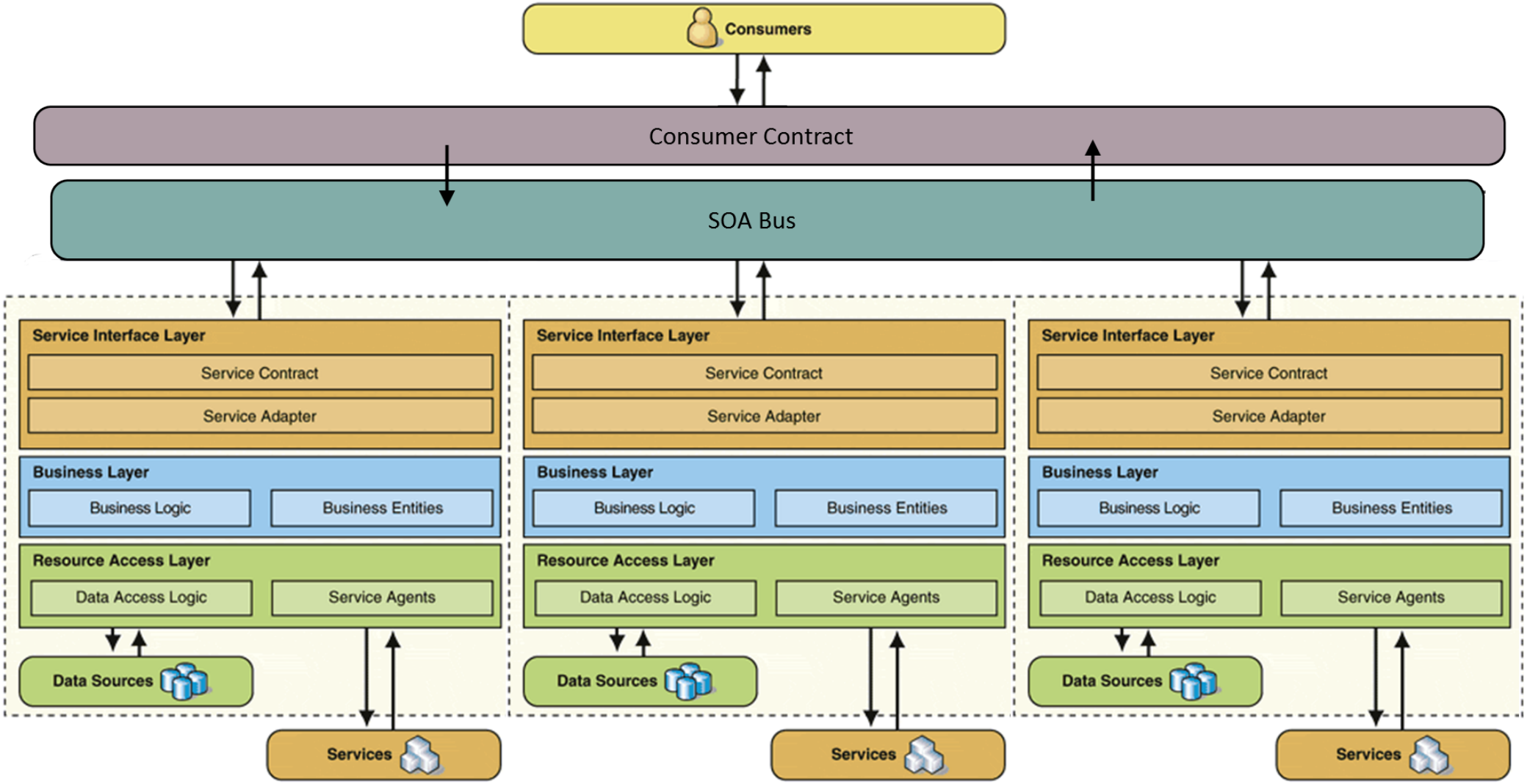
<u>This could be through a Service Oriented Architecture or a service existing on its own.</u>

- A service could be “make a reservation at Hotel”
  - The service exposes an interface which can be understood by all callers
  - The caller doesn’t know the underlying implementation, they just care about the interface
  - The service may use a number of machines/applications/functions to make the reservation (the caller doesn’t care)

- If the reservation database server is old it can be replaced
  - If the server has a new implementation, the service must be updated
  - None of the other components that rely on the database have to be changed

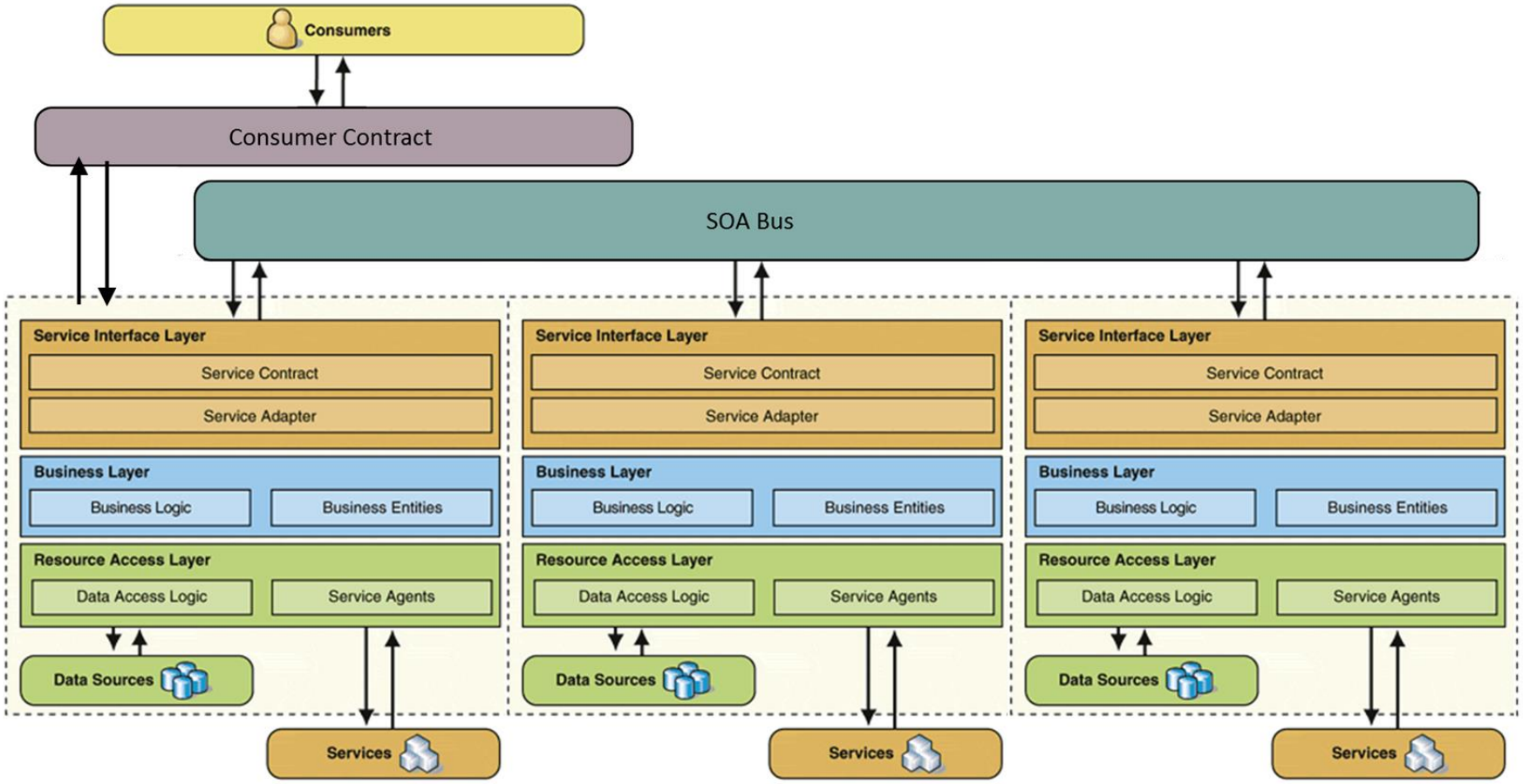
UNIVERSITY of HULL

SOA Layered Architecture



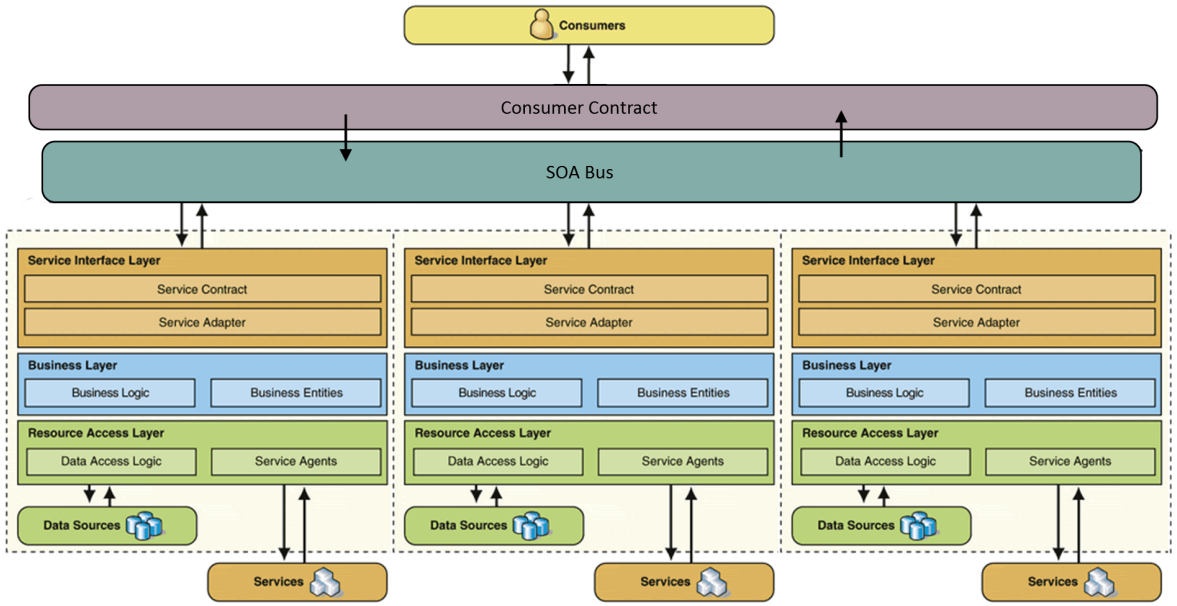
SOA Layered Architecture

Remember these sorts of architectures aren't a 'one-size-fits-all'
Perhaps the consumer interacts with only one service rather than onto the bus….



Benefits of a SOA

- Helps businesses react quickly and cost-effectively to a changing market.
  - Quickly swap out components.
- Components can be reused on a macro-level not on a micro-level (i.e. services are reused, not classes)
- Communication over a common bus improves internal efficiency
- Contracts enforce clear behaviours for communication.



UNIVERSITY of HULL

Contracts

- Traditional SOA wants explicit boundaries to bus
- Contracts specify what and how information crosses boundaries
  - Contract forms a public interface
  - Make interface easy to consume
  - Allow change to interface without breaking current consumers (backwards compatibility)
  - Avoid RPC (don’t try to call methods remotely - use messages to improve loose coupling)
  - Don’t expose implementation details

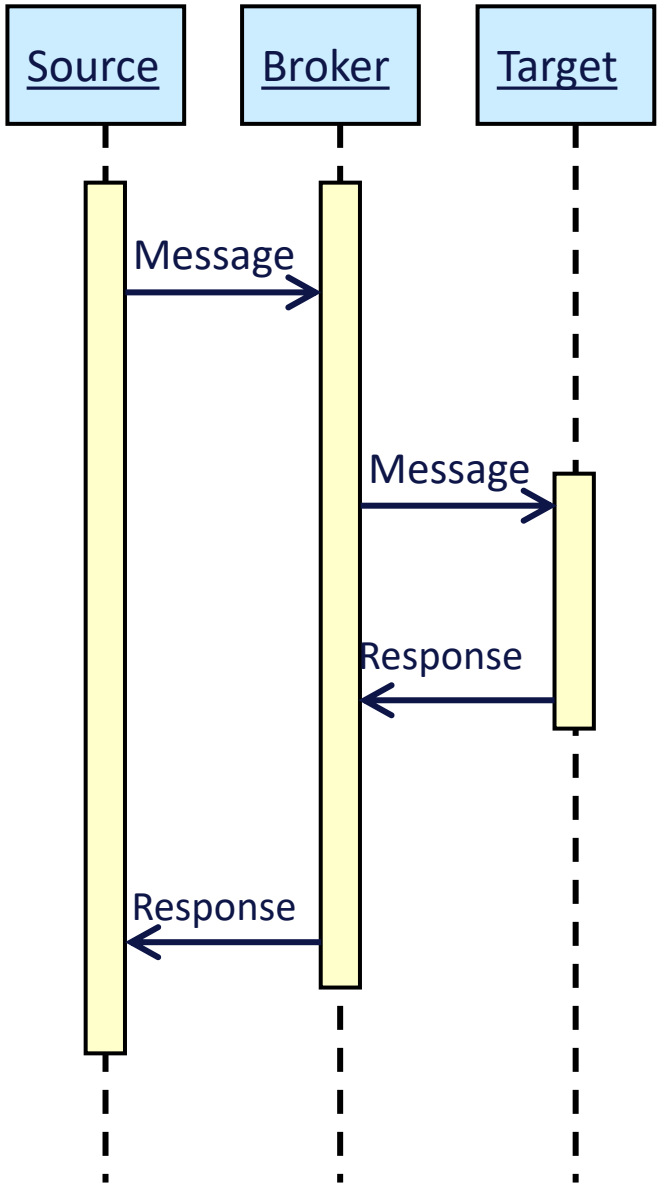
Contracts

- Service contract (service interface)
  - Declares exposed available behaviours (methods)
- Data contract
  - Declares data types to be sent (either way) across the boundary
- Message contract
  - Declares acceptable structure of message(s)

With this information we know how to use the service and can manually or programmatically generate a proxy to communicate with the service.

Roles in a SOA

- Service provider:
  creates and registers a service (e.g. with indirect broker)
  provides (a well defined) interface
  allows communication e.g. via messaging
  trade-off to consider: secure vs easy access
- Service consumer:
  accesses services (e.g. via indirect broker)
  can access multiple services (at once)
  needs to listen to / receive messages
- Service bus / broker:
  coordinates messages
  Can implement BUS and BROKER pattern simultaneously.



Micro-Service Architectures

Micro-Service Architectures are a subset of Service Oriented Architectures

- They have been around a long time but have become fashionable because of new opportunities afforded to us by Cloud Computing and Containerisation (more on that in a future lecture)

These architectures have the same principles as SOA but each service has a smaller, more explicitly defined purpose.

- A SOA collects together large systems, comprised of multiple components, which are each exposed as a service through the common bus
- A Microservice-Architecture collects together multiple single-function components which are independent and usually coordinated by a gateway (which often implements a bus)

UNIVERSITY of HULL

Micro-Service Architectures

Example services vs. micro-services

Micro-Services	Services
Authentication service	Website service
Logging service	Email service
Data retrieval service	API service
Data storage service	Database service
Verification service	Streaming service
Search service	Storage service

UNIVERSITY of HULL

Micro-Service Architectures

The major **benefits** of Micro-Service Architectures are:

- Scalability… A new instance of each service can be started when needed. This could be:
  - in response to a request which needs to use the service or
  - when the number of users increase and one service can't manage the load.
- The key thing is that not every service needs to scale at the same rate

- If one service fails, the others can remain functional.
  - Only a minor loss of functionality (unless the single service is integral, e.g. auth)

- Services can be created and maintained in isolation
  - easier to create (as a team)
  - easier to maintain, update, replace
  - better abstraction
  - new services can be quickly added

UNIVERSITY of HULL

Micro-Service Architectures

The major **drawbacks** of Micro-Service Architectures are:

- Knowing what will happen with a message can be complex from an outside perspective:
  - which service(s) use the message and what do they do with it?
  - is there a service pipeline (e.g. service X needs to manipulate the message before service Y uses it)?
- There is the possibility of fault hiding:
  - Service A is working properly so sends a normal-looking response
  - Service B has stopped working so does not log the interaction
- With one service managing both functionalities it either works or does not
- Can reduce performance, especially when:
  - Many services have to pass messages between themselves
  - Services only start when required (start-up time)

UNIVERSITY of HULL

How to achieve **loose coupling** with SOA

- **Interfaces:** a small set of well-defined interfaces that are known to consumers and providers – use of service contracts, use of facades
- **Schemas:** messages conform to a schema (message contract) that defines the structure of messages; this remains stable across updates
- **Extensibility:** architecture needs to be extensible and allow additions of new services and messages without breaking old components
- **Autonomous components:** allows replacement, improvement and testing of individual components (as long as interfaces and messages remain stable); separation of concerns.

UNIVERSITY of HULL

How to achieve **loose coupling** with SOA

- **Services should allow easy integration:** use persistent and well-defined interfaces and messaging conventions (based on industry standards and agreed data formats)

- **Services need to be "stateless":** ideally, a message should contain all necessary information for easy processing
  - This isn't always possible though: sessions may need to be established between service and consumer (for security reasons, convenience of a customised service, etc.)
  - This isn't always required either: sessions allow a service to 'remember' a consumer to increase performance… but at the cost of lower scalability.

UNIVERSITY of HULL

Web Services

- Often, when people are talking about Services they are referring to…

- Web Services… now a more generic term but we'll be using it to define a service:
  - Using the Simple Object Access Protocol (SOAP)
  - And standardised interfaces (e.g. defined in a Web Service Description Language (WSDL))
  - Sometimes implemented using (e.g.) Windows Communication Foundation (WCF)

- RESTful Services (we'll take a look at this in a future lecture)
  - Representational State Transfer (not a protocol)
  - Following the rules of HTTP / HTTPS
  - May be (but usually isn't) using an interface (e.g. in Web Application Description Language (WADL))

- Real-Time Services (we'll take a look at this in a future lecture)
  - E.g. SignalR – We'll investigate this in the labs

All of these *could* be part of a larger SOA, or exist independently

UNIVERSITY of HULL

A Note on Contracts and WSDL

The next slides consider SOAP contracts and Web Services Description Language (WSDL).

This is key theory because it is used, especially in enterprise environments.

However... Many newer technologies, frameworks and companies shun the use of contracts because of the additional overheads and the limited benefits that contracts give you.

There **are still** benefits though:

- Contracts allow very clear and unambiguous principles which define the services. These contracts can be defined and agreed ahead of time.
- Standardisation can facilitate cross-framework interoperability and allow consistency
- Understanding the benefits and drawbacks can help us to pick and choose mechanisms that will work for our own solutions

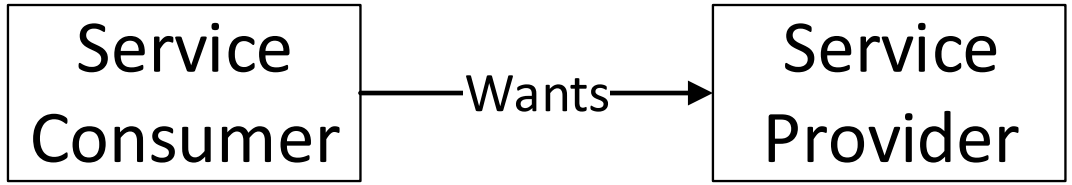
UNIVERSITY of HULL

Web Services – How?

- A service consumer wants a service over the web

This service is common (e.g. a flight reservation service) so lots of businesses offer it

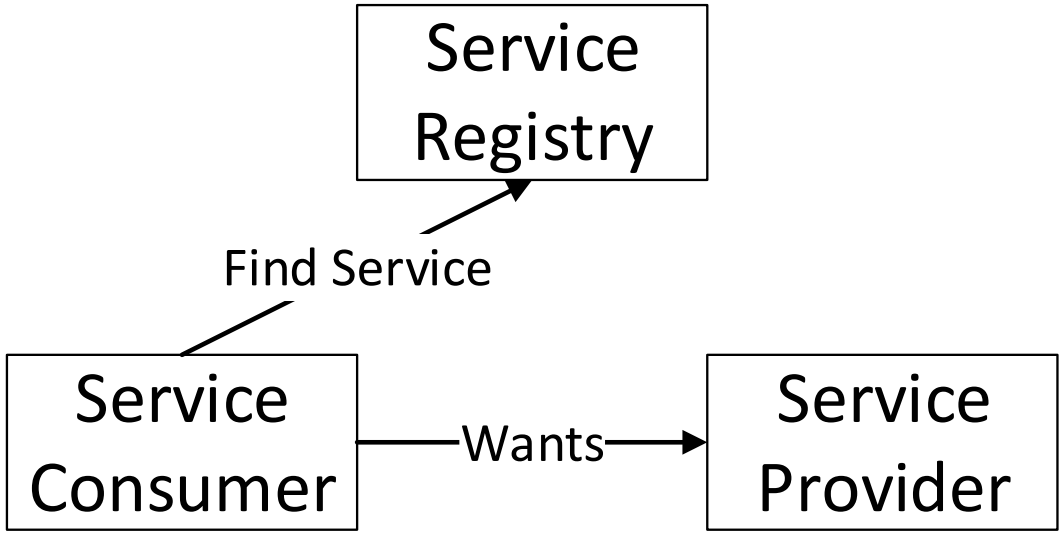
It therefore makes sense for it to be standardised…



UNIVERSITY of HULL

Web Services – How?

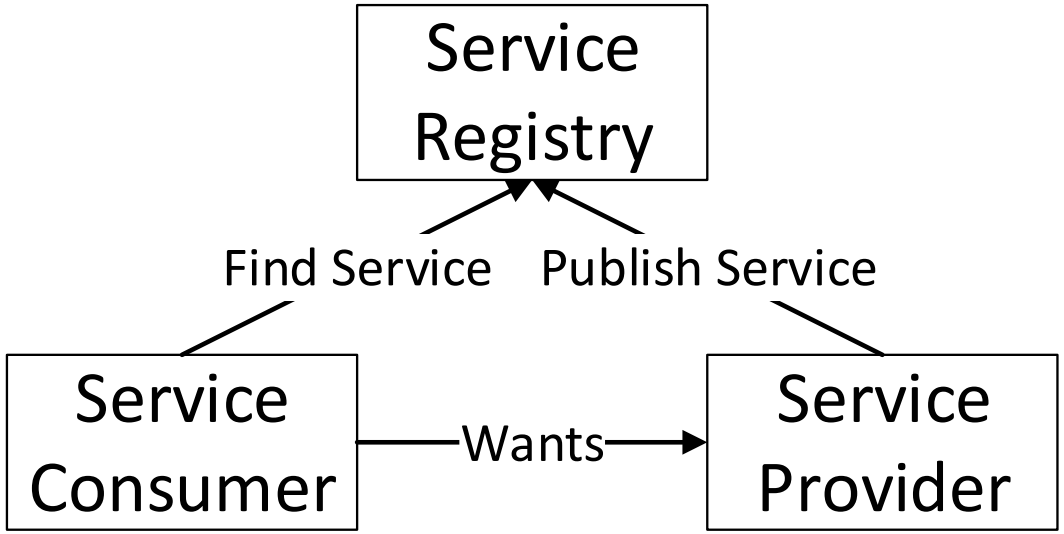
- A service consumer wants a service over the web
- It doesn't necessarily know where that provider is so it must look this up
- A Universal Description, Discovery and Integration Standard (UDDI) exists for this type of service which is standardised across the industry
- The consumer searches a UDDI registry (it's like a phone book) to find the service it wants



UNIVERSITY of HULL

Web Services – How?

- Before the registry knows where the provider is, the provider must publish itself to the registry
- It sends with its registration a description of its service endpoints using Web Services Description Language (WSDL)



UNIVERSITY of HULL

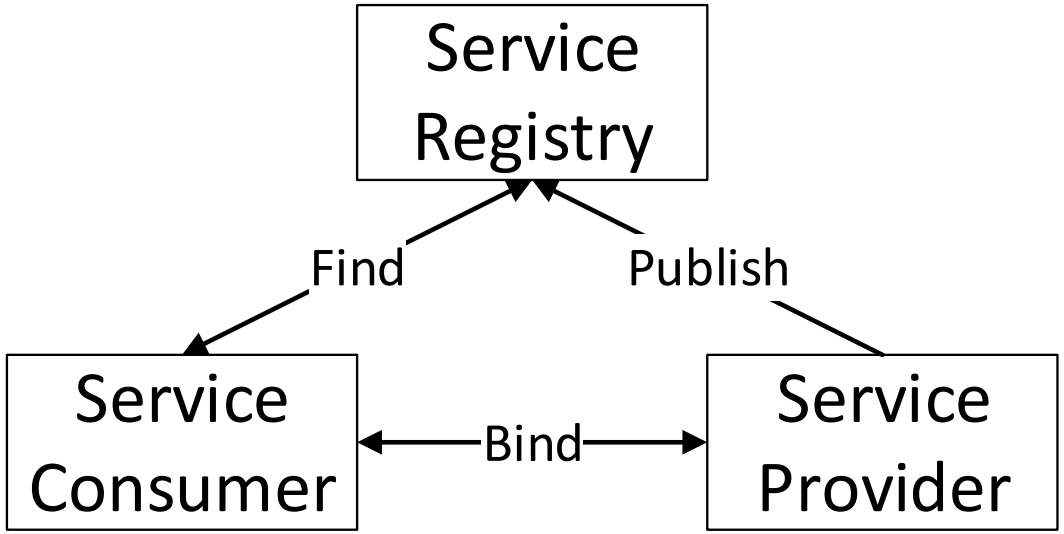
Web Services – How?

- Now, the registry can tell the consumer where the service is…

…And send it information about the service endpoints it can use (including the data types that should be sent/received)…

…So the service can bind itself directly to the provider and use the service as intended

Service registry acts as a Direct Broker!



UNIVERSITY of HULL

Web Service Fundamentals

Web services are based on open standards, e.g…

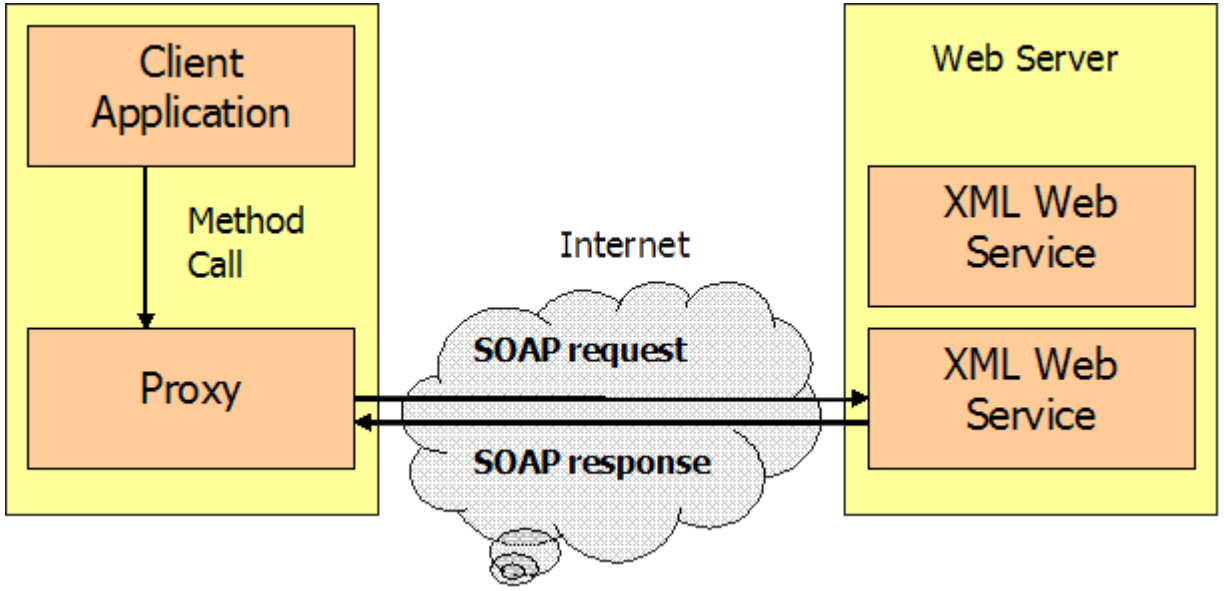
- Web Services Description Language (WSDL) – Describes '**how** to talk to the service'
  - XML description of Web service **interface** (contract)
  - Used by consumers, set by providers
  - Describes web service: name, parameters, return values

- Simple Object Access Protocol (SOAP) – Message format for sending/receiving
  - Format is defined in the WSDL contract
  - XML encoding of data types
  - Used to serialize arguments and return values
  - Designed as a standard to make it easy to deserialize at the other end

UNIVERSITY of HULL

Web Service Architecture

- Client uses the info in the WSDL to create its proxy during *development*
  - Often automated once WSDL is identified
- At runtime, client application uses proxy to interact with Web service
- Proxy handles marshalling (transforming data for transmission) and other low-level details
- Developer can focus on application specifics and keeps business layer away from service interface layer.



UNIVERSITY of HULL

Programming with SOAP

- Service
  - Class **implementing** a WSDL (Web Service Description Language) interface
  - Sends/recieves information in messages as per the WSDL

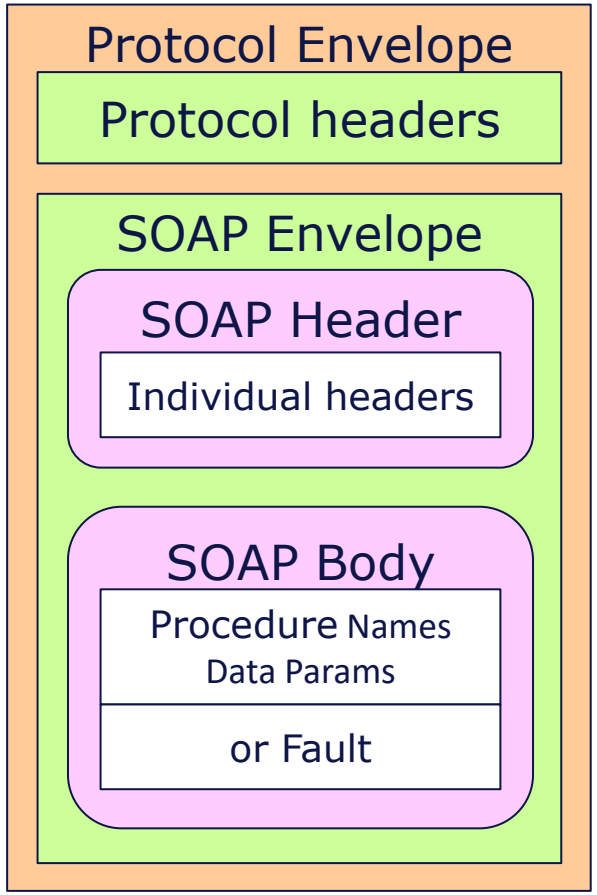
- Message
  - Data transfer object comprising header and body
  - Header contains information about the message's transmission
  - Body contains the actual data
  - When message arrives, data may be converted into objects
    - These need not be the same as the objects from which it came
    - This is **Loose Coupling**, and it's a good thing!

UNIVERSITY of HULL

SOAP

- Standardised XML schema for representing:
  - Headers (e.g. data encoding)
  - Remote procedure calls
  - Parameters (data) to be sent to remote procedures
- Extensible
  - Easy to add, modify and remove custom procedures/parameters
- Network, platform and language agnostic
- Supported by many web service frameworks

- Often transported on HTTP but could use any protocol



UNIVERSITY of HULL

WSDL – Web Service Definition Language

- WSDL documents specify public interface to web services (**contract**)
  - Perfect for an SOA because this allows all of the services to be registered with the bus using the same language (even when the services are all programmed in different languages)

- Specifes definitions, operations and service bindings
  - <portType>
    - Defines operations performed by the service
  - <message>
    - Defines what should be contained in the message itself
  - <types>
    - Defines any custom data types consumed or delivered
  - <binding>
    - Allows us to bind the WSDL to specific operations defined in SOAP (through serialisation)
    - Specifies how the services are called (e.g. use HTML)

UNIVERSITY of HULL

WSDL – Example

```
<portType name="myThings">
  <operation name="getThing">
    <input message="getThingRequest"/>
    <output message="getThingResponse"/>
  </operation>
</portType>
```

Definition of all operations which can be performed within the myThings portType – here, only one operation: getThing.

```
<message name="getThingRequest">
  <part name="thing" type="xs:string"/>
</message>
```

What the request Message should contain (name and data type)

```
<message name="getThingResponse">
  <part name="value" type="xs:string"/>
</message>
```

What the response Message will contain (name and data type)

```
<binding type="myThings" name="myBinding">
   <soap:binding style="document"
   transport="http://schemas.xmlsoap.org/soap/http" />
   <operation>
     <soap:operation soapAction="http://mydomain.com/getThing"/>
     <input><soap:body use="literal"/></input>
     <output><soap:body use="literal"/></output>
  </operation>
</binding>
```

A binding which links the WSDL portType to specific SOAP endpoints and defines where the data is to be expected in the message (body)

UNIVERSITY of HULL

Disadvantages of Web Services (SOAP and WSDL)

- Data bloat
  - Formatting data in XML adds lots of overhead
  - Often we are just passing data – all of the remote procedure information is redundant
  - Can get around this to some degree by compression

- Services tend to be expected to be able to speak to all service requestors
  - This doesn't work well if your service has been created to use data or functionality that is (for example) relevant to Windows only.

- Do we always need to advertise exactly how our services can be used at runtime?
  - Developers often don't write software that needs to find and work out how to use services at runtime – often this happens during development!

- HTTP already does lots of what Web Services does for us – unnecessary complexity

And again…

SOAP is used rarely nowadays but still **is** used, especially in business and legacy systems

It is a particularly important example for examining contract-based protocols as there are few common strict contract, human-readable protocols in use today

Where RPC is required there are other significantly more efficient techniques.

The disadvantage of losing this level of contractual assurance is that when things don’t work you can’t check the contract to see what you might be doing wrong.

UNIVERSITY of HULL

Reading

Architecture: Analysis and Design for Services and Microservices

Thomas Erl
Prentice Hall, 2017

This book is useful because it discusses both services and microservices together

UNIVERSITY of HULL

Reflection

In this session we've looked at:

- Services
- Service Oriented Architectures (SOA)
- Services in Business Applications
- Web Services
- The Web Service architecture

In the next session we will look at Resource Oriented Architectures (ROA) and Representational State Transfer (REST)

UNIVERSITY of HULL